

Ataque a la Pila por *Stack Smashing* en Linux i386

V. Blanco, A. Navarro, J. Ferreira & T. Oganessian

22 de junio de 2026

1 Resumen

Esta memoria está dedicada a analizar un ataque por *stack smashing* en un sistema Linux. Concretamente, la víctima es un sistema para procesadores i386, lo que será importante en cuanto al lenguaje ensamblador a analizar.

Para ello, se ha hecho uso del lenguaje de programación C, la herramienta *make* y el compilador y debugger de GNU, todo bajo un subsistema Docker.

2 Código Vulnerable

La vulnerabilidad explotada es un *buffer overflow* que se da por la naturaleza del código. Las siguientes líneas de la función `smash()` (archivo `stsmash.c`) son clave:

```
char buffer[500];
read(0, buffer, 700);
```

Como se puede ver en la declaración del buffer y la llamada a la función `read()`, se permite que el usuario introduzca más datos de los que el buffer puede alojar.

3 Metodología de ataque

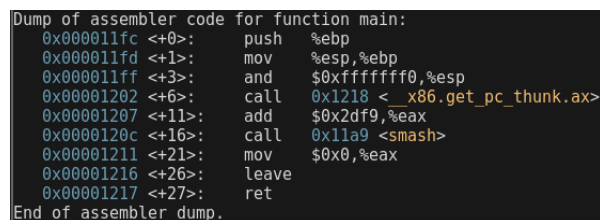
El ataque pretende conseguir desbordar el buffer alojado en el stack. Para ello, se aprovecha el hecho de que la función `read()` no comprueba límites en el buffer de ninguna manera.

Conociendo el código, es fácil darse cuenta de que, como $700 > 500$, podemos sobrepasar el límite superior del buffer. Sin embargo, si se considerara el programa como una caja negra (sin tener acceso al código), habría que realizar un análisis profundo, descompilando el programa.

4 Análisis con GDB

Para realizar el análisis real del código a partir del binario compilado, utilizamos el debugger de GNU (GDB) y algunas instrucciones.

Comenzamos desensamblando las funciones del programa. Todo programa a bajo nivel tiene una función principal, `main()`. Empezaremos examinando dicha función. Véase la Figura 1.



```
Dump of assembler code for function main:
0x000011fc <+0>:  push    %ebp
0x000011fd <+1>:  mov     %esp,%ebp
0x000011ff <+3>:  and     $0xffffffff,%esp
0x00001202 <+6>:  call    0x1218 <_x86.get_pc_thunk.ax>
0x00001207 <+11>: add     $0x2df9,%eax
0x0000120c <+16>: call    0x11a9 <smash>
0x00001211 <+21>: mov     $0x0,%eax
0x00001216 <+26>: leave   %eax
0x00001217 <+27>:  ret
End of assembler dump.
```

Figura 1: Desensamblado de la función `main`.

A partir de este código ensamblador, descubrimos una cierta función de nombre *smash*. La desensamblamos. Véase la Figura 2.

6 Mitigaciones

Los compiladores de hoy en día presentan multitud de herramientas y niveles de análisis de código internos para tratar de evitar este tipo de ataques.

Por ejemplo, cuentan con el ASLR (*Address Space Layout Randomisation*), que, en base a aleatorizar las zonas de memoria donde se ejecuta un programa cada vez, puede evitar accesos directos a ciertas partes del sistema [1]; o la flag `-D_FORTIFY_SOURCE`, desarrollada por Red Hat para la GCC, que intenta evitar la ejecución de código inyectado a partir de cadenas de caracteres [2].

7 Conclusión

Aunque la mayor parte de los sistemas instalados hoy en día sean fuertes o casi invulnerables a este tipo de ataque, no hay que olvidar que existen grandes cantidades de servidores utilizando versiones desactualizadas paquetes de compiladores y el kernel de Linux. Por tanto, esta es una vulnerabilidad que los equipos azules de, sobre todo, empresas y organizaciones grandes deben de tener en cuenta a la hora de ejecutar programas en sus servidores.

Referencias

- [1] Wikipedia Contributors, *Address space layout randomization*, https://en.wikipedia.org/w/index.php?title=Address_space_layout_randomization&oldid=1355882436, (Accesed: June 22, 2026).
- [2] Wikipedia Contributors, *Exec Shield*, https://en.wikipedia.org/w/index.php?title=Exec_Shield&oldid=1268856958, (Accessed: June 22, 2026).